

# Algorithms

Lecture #04

**Syed Hamed Raza**

# Running Time Analysis

## ***Running Time Analysis***

- The running time depends upon the input size, e.g.  $n$
- Different inputs of the same size may result in different running time.
- For example, breaking out of the inner loop in the brute-force algorithm depends not only on the input size of  $P$  but also the structure of the input.

# Running Time Analysis: Criteria

## ***Running Time Analysis: Criteria***

Two criteria for measuring running time:

### **1. Worst-case time:**

- Worst-case time is the maximum running time over all (legal) inputs of size  $n$ .

# Running Time Analysis: Criteria

## *Running Time Analysis: Criteria*

### **Worst-case time:**

- Let  $I$  denote an input instance,
- let  $|I|$  denote its length, and
- let  $T(I)$  denote the running time of the algorithm on input  $I$ .
- Then

$$T_{\text{worst}}(n) = \max_{|I|=n} T(I)$$

# Running Time Analysis: Criteria

## ***Running Time Analysis:*** ***Criteria*** **2. Average-case time**

- Average-case time is the average running time over all inputs of size  $n$ .

# Running Time Analysis: Criteria

## ***Running Time Analysis: Criteria***

### **2. Average-case time**

- Let  $p(I)$  denote the probability of seeing this input.
- The average-case time is the weighted sum of running times with weights being the probabilities:

$$T_{\text{avg}}(n) = \sum_{|I|=n} p(I)T(I)$$



# Running Time Analysis

## ***Running Time Analysis***

- We will almost always work with worst-case time.
- Average-case time is more difficult to compute; it is difficult to specify probability distribution on inputs.
- Worst-case time will specify an upper limit on the running time.

# Analysis of 2-dimension maxima

## ***Analysis of 2-dimension maxima***

Analysis of the brute-force maxima algorithm.

- Input size is  $n$
- We will count number of times that any element of  $P$  is accessed.



# Analysis of 2-dimension maxima

## ***Example: 2-dimension maxima***

```
MAXIMA(int n, Point P[1 . . . n])  
1  for i ← 1 to n      n times  
2  do maximal ← true  
3    for j ← 1 to n    n times  
4    do  
5      if (i ≠ j) & (P[i].x ≤ P[j].x) &  
6        (P[i].y ≤ P[j].y) 4 accesses  
        then maximal ← false; break
```

# Analysis of 2-dimension maxima

## ***Analysis of 2-dimension maxima***

- The outer loop runs  $n$  times
- For each  $i$  iteration, the inner loop runs  $n$  time.
- $P$  is accessed four times in the *if* statement.
- The output statement accesses  $P$  2 times.
- In the worst case, every point is maximal so every point is output.

## **Example: 2-dimension *maxima***

```
2  do maximal ← true
3    for  $j \leftarrow 1$  to  $n$   n times
4    do
5      if  $(i \neq j) \ \& \ (P[i].x \leq P[j].x) \ \&$ 
6         $(P[i].y \leq P[j].y)$   4 accesses
7      then maximal ← false; break
8    if maximal
9      then output  $P[i].x,$ 
10                $P[i].y$   2 accesses
```

# Example: 2-dimension maxima

## ***Example: 2-dimension maxima***

```
MAXIMA(int n, Point P[1 . . . n])  
1  for i ← 1 to n      n times  
2  do maximal ← true  
3    for j ← 1 to n    n times  
4    do  
5      if (i ≠ j) & (P[i].x ≤ P[j].x) &  
6        (P[i].y ≤ P[j].y)  4 accesses  
        then maximal ← false; break
```

# Analysis of 2-dimension maxima

## ***Analysis of 2-dimension maxima***

- The outer loop runs  $n$  times
- For each  $i$  iteration, the inner loop runs  $n$  time.
- $P$  is accessed four times in the *if* statement.
- The output statement accesses  $P$  2 times.
- In the worst case, every point is maximal so every point is output.

# Analysis of 2-dimension maxima

## ***Analysis of 2-dimension maxima***

Worst-case running time:

Pair of nested summations, one for i-loop and the other for the j-loop

$$T(n) = \sum_{i=1}^n \left( 2 + \sum_{j=1}^n 4 \right)$$



# Analysis of 2-dimension maxima

## ***Analysis of 2-dimension maxima***

Worst-case running time:

$$T(n) = \sum_{i=1}^n \left( 2 + \sum_{j=1}^n 4 \right)$$

$$\left( \sum_{j=1}^n 4 \right) = 4n, \text{ and so}$$

$$\begin{aligned} T(n) &= \sum_{i=1}^n (4n + 2) \\ &= (4n + 2) \cdot n = 4n^2 + 2n \end{aligned}$$

# Analysis of 2-dimension maxima

## ***Analysis of 2-dimension maxima***

- For small values of  $n$ , any algorithm is fast enough.
- What happens when  $n$  gets large?
- Running time does become an issue.
- When  $n$  is large,  $n^2$  term will be much larger than the  $n$  term and will **dominate** the running time.

# Analysis of 2-dimension maxima

## ***Analysis of 2-dimension maxima***

- We will say that the worst-case running time is  $\Theta(n^2)$ .
- This is called the *asymptotic growth rate* of the function.
- We will discuss this  $\Theta$ -notation more formally later.

# Summations

## *Summations*

- The analysis involved computing a summation.
- Summation should be familiar but let us review a bit here.

# Summations

## *Summations*

- Given a finite sequence of values  $a_1, a_2, \dots, a_n$ ,
- their sum  $a_1 + a_2 + \dots + a_n$  is expressed in **summation notation** as

$$\sum_{i=1}^n a_i$$

- If  $n = 0$ , then the sum is additive identity, 0.

# Summations

## *Summations*

Some facts about summation:

- If  $c$  is a constant

$$\sum_{i=1}^n ca_i = c \sum_{i=1}^n a_i$$

and

$$\sum_{i=1}^n (a_i + b_i) = \sum_{i=1}^n a_i + \sum_{i=1}^n b_i$$



# Summations

## *Summations*

Some important summations that should be committed to memory.

Arithmetic series:

$$\begin{aligned}\sum_{i=1}^n i &= 1 + 2 + \dots + n \\ &= \frac{n(n+1)}{2} = \Theta(n^2)\end{aligned}$$

# Summations

## *Summations*

Quadratic series:

$$\begin{aligned}\sum_{i=1}^n i^2 &= 1 + 4 + 9 + \dots + n^2 \\ &= \frac{2n^3 + 3n^2 + n}{6} = \Theta(n^3)\end{aligned}$$

# Summations

## *Summations*

Geometric series:

$$\sum_{i=1}^n x^i = 1 + x + x^2 + \dots + x^n$$
$$= \frac{x^{(n+1)} - 1}{x - 1} = \Theta(n^2)$$

If  $0 < x < 1$  then this is  $\Theta(1)$ , and  
if  $x > 1$ , then this is  $\Theta(x^n)$ .

# Summations

## *Summations*

Harmonic series: For  $n \geq 0$

$$\begin{aligned} H_n &= \sum_{i=1}^n \frac{1}{i} \\ &= 1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n} \approx \ln n \\ &= \Theta(\ln n) \end{aligned}$$

# A harder Example

## *A Harder Example*

NESTED-LOOPS()

1 for  $i \leftarrow 1$  to  $n$

2 do

3   for  $j \leftarrow 1$  to  $2i$

4    do  $k = j \dots$

5      while  $(k \geq 0)$

6      do  $k = k - 1 \dots$

How do we analyze the running time of an algorithm that has complex nested loop?

# A harder Example

## ***A Harder Example***

NESTED-LOOPS()

1 for  $i \leftarrow 1$  to  $n$

2 do

3   for  $j \leftarrow 1$  to  $2i$

4    **do**  $k = j \dots$

5       **while** ( $k \geq 0$ )

6       **do**  $k = k - 1 \dots$

To convert loops into summations, we work from inside-out.



# A harder Example

## *A Harder Example*

NESTED-LOOPS()

```
1 for i ← 1 to n
2 do
3   for j ← 1 to 2i
4     do k = j . . .
5       while (k ≥ 0)
6         do k = k - 1 . . .
```

The answer is we write out the loops as summations and then solve the summations.

# A harder Example

## ***A Harder Example***

NESTED-LOOPS()

```
1 for i ← 1 to n
2 do
3   for j ← 1 to 2i
4     do k = j . . .
5       while (k ≥ 0)
6         do k = k - 1 . . .
```

The answer is we write out the loops as summations and then solve the summations.

# A harder Example

## ***A Harder Example***

NESTED-LOOPS()

```
1 for i ← 1 to n
2 do for j ← 1 to 2i
3   do k = j
4     while (k ≥ 0) 
5     do k = k - 1
```

- Consider the inner most *while* loop.
- It is executed for  $k=j, j-1, j-2, \dots, 0$ .

# A harder Example

## *A Harder Example*

NESTED-LOOPS()

```
1 for i ← 1 to n
2 do for j ← 1 to 2i
3   do k = j
4     while (k ≥ 0) 
5     do k = k - 1
```

- Time spent inside the while loop is constant.
- Let  $I()$  be the time spent in the while loop.

Thus:

$$I(j) = \sum_{k=0}^j 1 = j + 1$$

# A harder Example

## ***A Harder Example***

NESTED-LOOPS()

1 **for**  $i \leftarrow 1$  **to**  $n$

2 **do** **for**  $j \leftarrow 1$  **to**  $2i$  

3     **do**  $k = j$

4         **while**  $(k \geq 0)$

5         **do**  $k = k - 1$

- Consider the *middle* for loop.

- It's running time is determined by  $i$ .

# A harder Example

## *A Harder Example*

NESTED-LOOPS()

1 **for** i ← 1 **to** n

2 **do** **for** j ← 1 **to** 2i 

3     **do** k = j

4         **while** (k ≥ 0)

5         **do** k = k - 1

- Let M() be the time spent in the for loop:

$$M(i) = \sum_{j=1}^{2i} I(j)$$



# A harder Example

## ***A Harder Example***

$$\begin{aligned}M(i) &= \sum_{j=1}^{2i} I(j) = \sum_{j=1}^{2i} (j + 1) \\&= \sum_{j=1}^{2i} j + \sum_{j=1}^{2i} 1 \\&= \frac{2i(2i + 1)}{2} + 2i \\&= 2i^2 + 3i\end{aligned}$$

# A harder Example

## *A Harder Example*

NESTED-LOOPS()

```
1 for i ← 1 to n   
2 do for j ← 1 to 2i  
3   do k = j  
4   while (k ≥ 0)  
5   do k = k - 1
```

- Finally the outer-most for loop.
- Let  $T()$  be running time of the entire algorithm:

$$T(n) = \sum_{i=1}^n M(i)$$

# A harder Example

## ***A Harder Example***

$$\begin{aligned}T(n) &= \sum_{i=1}^n M(i) = \sum_{i=1}^n (2i^2 + 3i) \\&= \sum_{i=1}^n 2i^2 + \sum_{i=1}^n 3i \\&= 2 \frac{2n^3 + 3n^2 + n}{6} + 3 \frac{n(n+1)}{2} \\&= \frac{4n^3 + 15n^2 + 11n}{6} \\&= \Theta(n^3)\end{aligned}$$

# 2-dimension Maxima Revisited

## ***2-dimension Maxima Revisited***

Recall the 2-d maxima problem:

- Let a point  $p$  in 2-dimensional space be given by its integer coordinates,  $p = (p.x, p.y)$ .
- A point  $p$  is said to be dominated by point  $q$  if  $p.x \leq q.x$  and  $p.y \leq q.y$ .